
Week 3: High Level Design

Monday, June 2nd 2026

Topic Assigned

CAN Layer Architecture

Designing the UDS & ISO TP Architecture

UDS and ISO TP Architecture:

UDS Data Flow

1. UDS Request → ISO-TP → CAN → ECU

- **UDS Client Layer:** Sends request (e.g., `22 F1 90`).
- **UDS Protocol Layer:** Formats request as UDS message.
- **ISO-TP Layer:** Segments message into CAN frames (if >8 bytes).
- **CAN Interface:** Transmits frames over CAN bus (e.g., `0x7E0` for request).

2. ECU → CAN → ISO-TP → UDS Response → Application:

- **CAN Interface (ECU):** Receives frames (e.g., `0x7E0`).
- **ISO-TP Layer (ECU):** Reassembles frames into UDS message.
- **UDS Protocol Layer (ECU):** Processes request, generates response (e.g., `62 F1 90 AA BB CC`).
- **ISO-TP Layer (ECU):** Segments response into CAN frames (if >8 bytes).
- **CAN Interface (ECU):** Transmits response frames (e.g., `0x7E8`).
- **Client:** Receives and reassembles response, passes to application.

Examples:

- **Request:** `22 F1 90` (Service `0x22`: Read Data By Identifier, DID `0xF190`: VIN).
 - **Positive Response:** `62 F1 90 57 41 55 . . .` (`0x62` is the SID `0x22` + `0x40` indicating success, followed by the DID and the VIN ASCII data).
 - **Negative Response:** `7F 22 31` (`0x7F`: Error flag, `0x22`: Failed Service, `0x31`: Request Out Of Range).
-

Mermaid Codes

- **UDS Message Flow**

None

sequenceDiagram

participant Client

participant CAN_Bus

participant ECU

Client->>Client: UDS Request (e.g., 0x22 F190)

Client->>Client: ISO-TP Segmentation (if >8 bytes)

Client->>CAN_Bus: CAN Frames (Source: Client, Dest: ECU)

CAN_Bus->>ECU: Delivers Frames

ECU->>ECU: ISO-TP Reassembly

ECU->>ECU: UDS Processing (Read DID F190)

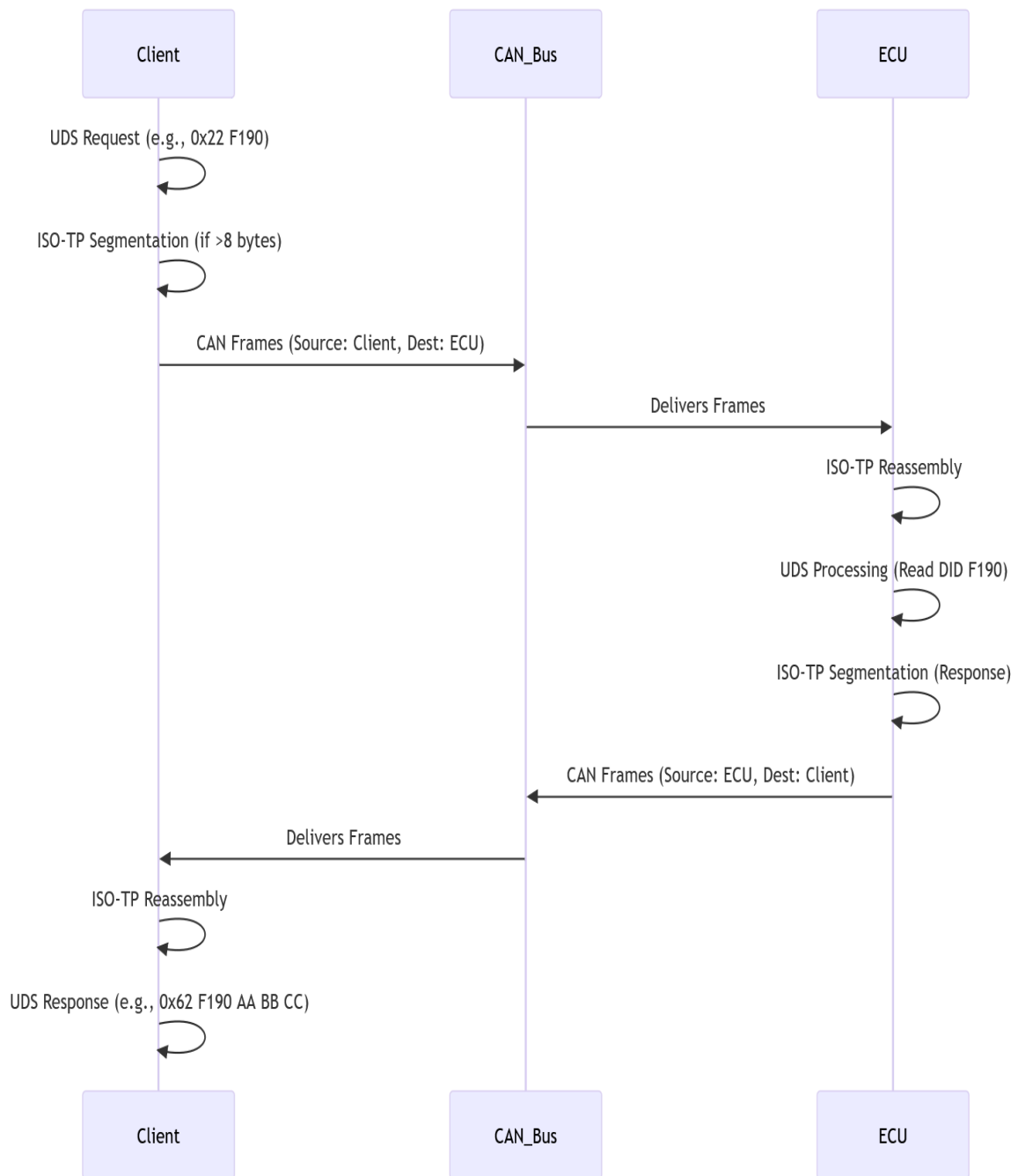
ECU->>ECU: ISO-TP Segmentation (Response)

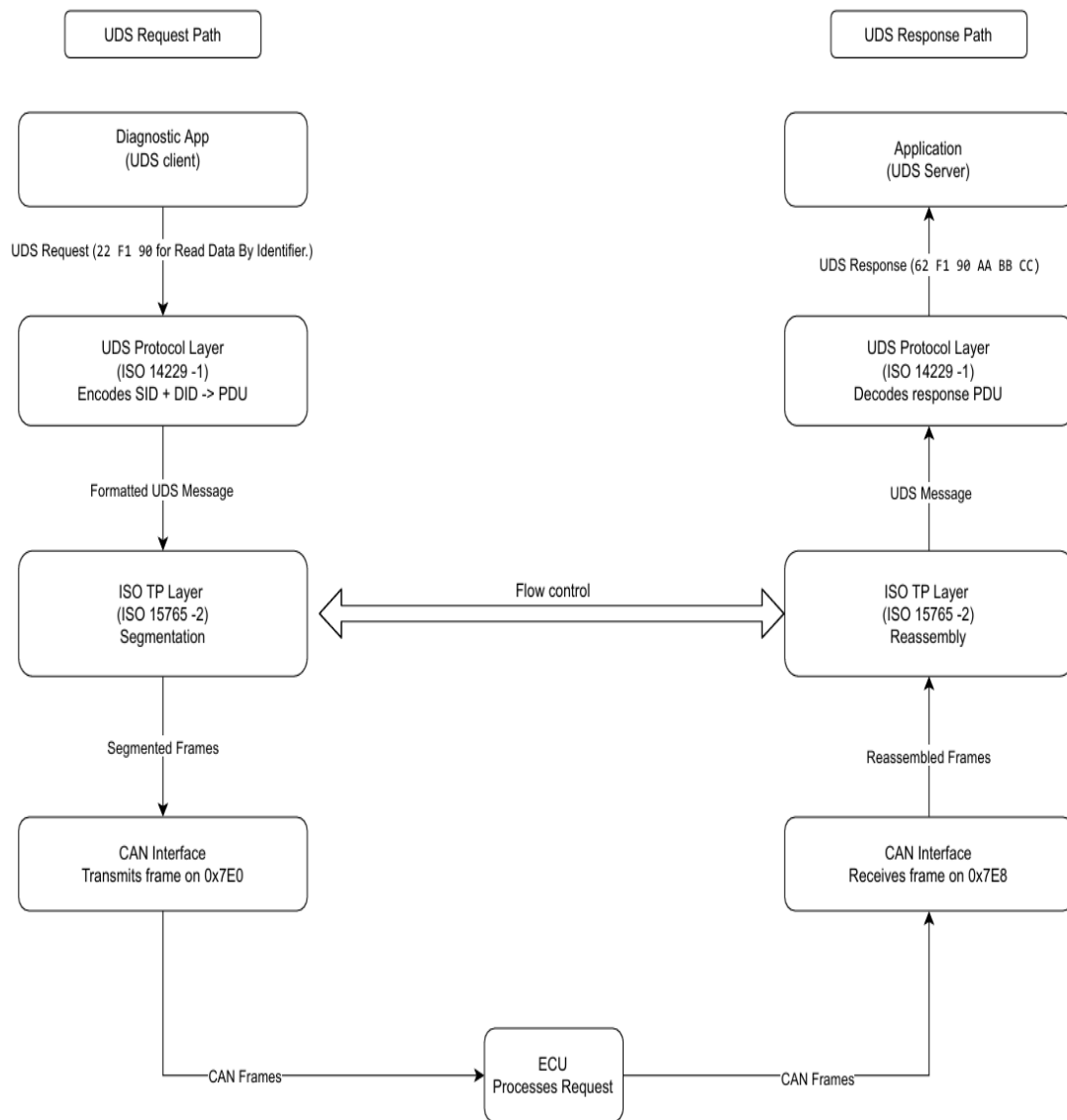
ECU->>CAN_Bus: CAN Frames (Source: ECU, Dest: Client)

CAN_Bus->>Client: Delivers Frames

Client->>Client: ISO-TP Reassembly

Client->>Client: UDS Response (e.g., 0x62 F190 AA BB CC)





• Security Diagram

Authorization & Authentication Message Flow

None

sequenceDiagram

participant Tester

participant ECU

Note over Tester, ECU: UDS Authentication using
Challenge-Response Mechanism
with Digital Signature
Verification (ISO 14229 SID 0x29)

Tester->>ECU: Authentication Request (SID=0x29)

Note right of ECU: Generate Random Challenge

ECU-->>Tester: Challenge = 0x7A 3F 92 D1 A4 BC 56 78

Note left of Tester: SHA-256 Hash Calculation

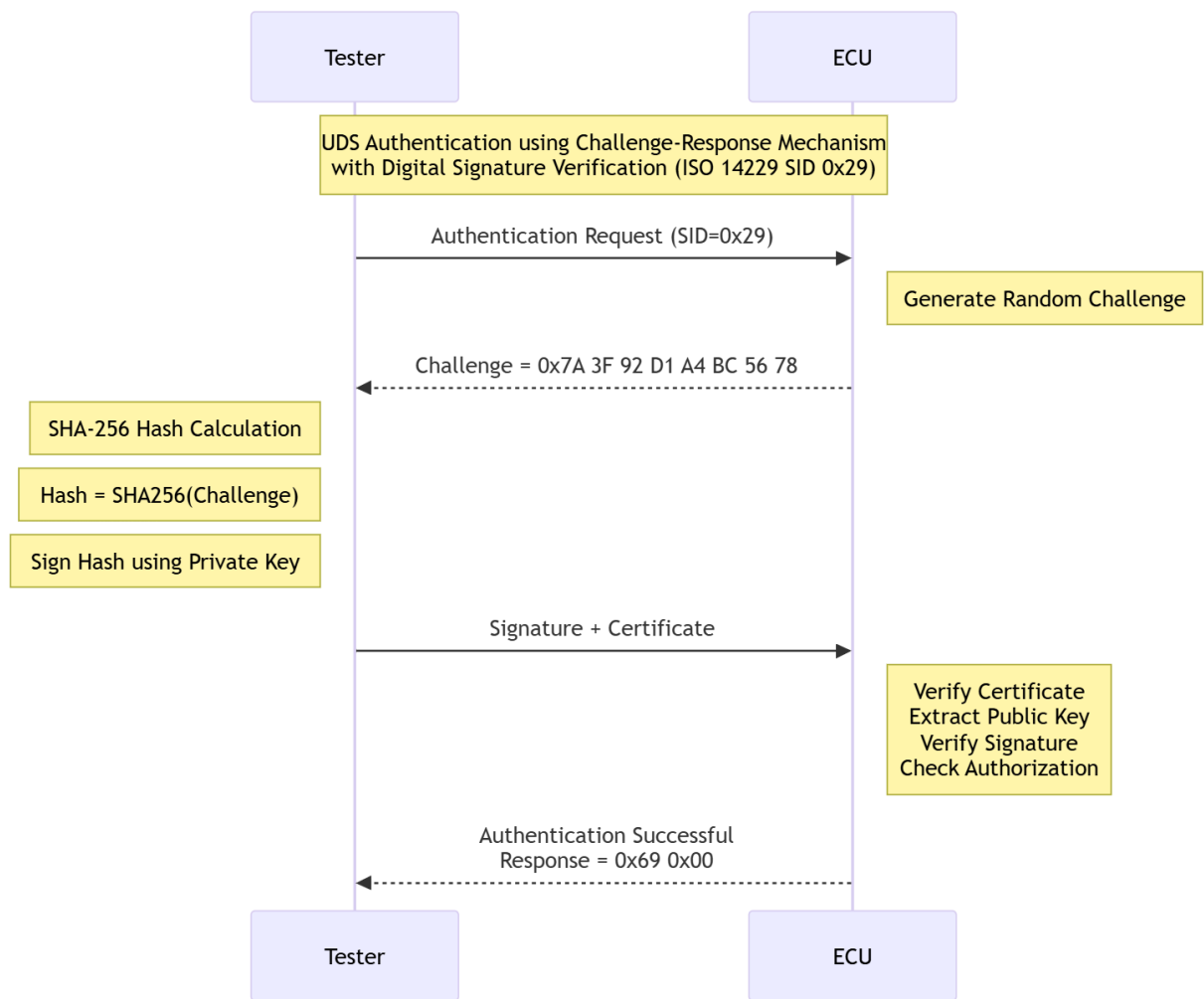
Note left of Tester: Hash = SHA256(Challenge)

Note left of Tester: Sign Hash using Private Key

Tester->>ECU: Signature + Certificate

Note right of ECU: Verify Certificate
Extract Public
Key
Verify Signature
Check Authorization

ECU-->>Tester: Authentication Successful
Response = 0x69
0x00



Authentication & Authorization In Depth

None

flowchart LR

A["Certificate + Signature"]

B["Parses the certificate
and extracts fields"]

C["Verifies OEM signature present on
Certificate by the root public key
already stored in ECU"]



```
D["Use the public key sent by tester to<br/>verify  
Signature<br/>verify(signature, public_key)"]
```

```
E["Identity Extraction<br/>from the certificate"]
```

```
F["Determines permission<br/>for the tester by using  
a<br/>lookup/policy table"]
```

```
G["Policy Evaluation"]
```

```
H["System creates trusted session<br/>context"]
```

```
I["Session:<br/>{<br/>Identity = DealerTool_123<br/>Role =  
Dealer<br/>Permissions =  
[<br/>ReadDTC,<br/>FlashECU,<br/>RunRoutine<br/>]<br/>SessionTimeou  
t = 15 min<br/>}"]
```

```
A --> B
```

```
B --> C
```

```
C --> D
```

```
D --> E
```

```
C --> F
```

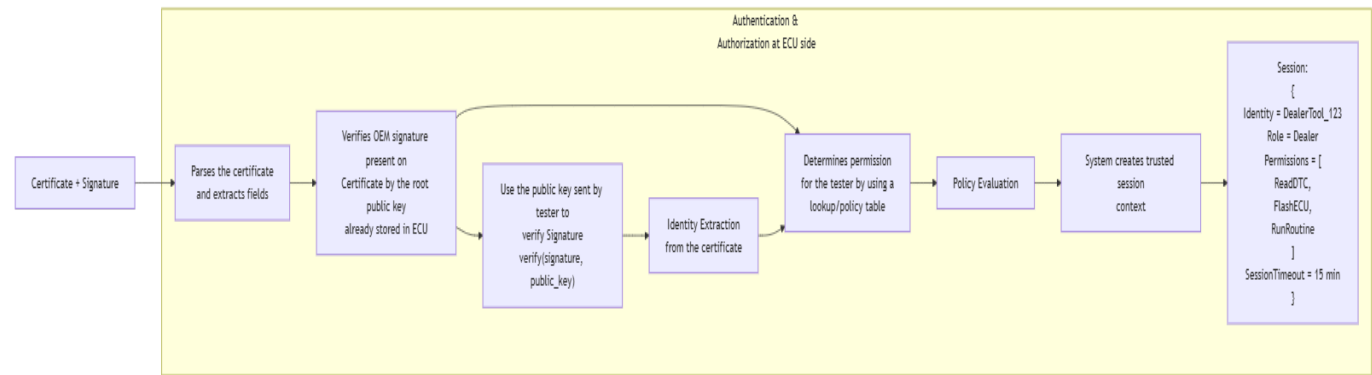
```
E --> F
```

```
F --> G
```

```
G --> H
```

```
H --> I
```

```
subgraph ECU["Authentication & Authorization at ECU side"]
    B
    C
    D
    E
    F
    G
    H
    I
end
```



Encryption

None

flowchart TB

```
subgraph Tester
    direction TB
```

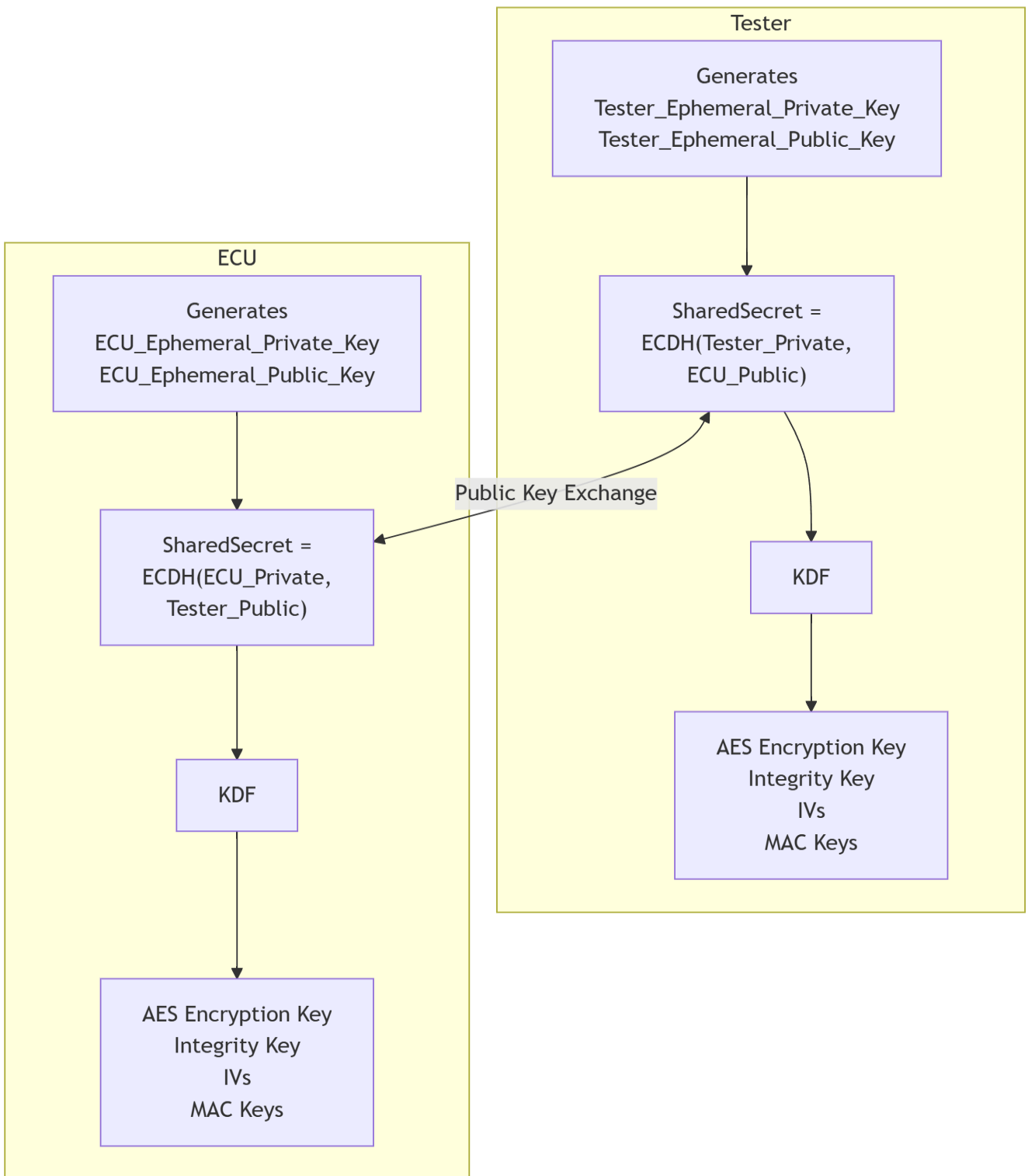


```

T1["Generates<br/>Tester_Ephemeral_Private_Key<br/>Tester_Ephemeral
_Public_Key"]
    T2["SharedSecret
=<br/>ECDH(Tester_Private,<br/>ECU_Public)"]
    T3["KDF"]
    T4["AES Encryption Key<br/>Integrity Key<br/>IVs<br/>MAC
Keys"]
    T1 --> T2
    T2 --> T3
    T3 --> T4
end
T2 <-->|Public Key Exchange| E2

subgraph ECU
    direction TB
    E1["Generates<br/>ECU_Ephemeral_Private_Key<br/>ECU_Ephemeral_Publi
c_Key"]
    E2["SharedSecret
=<br/>ECDH(ECU_Private,<br/>Tester_Public)"]
    E3["KDF"]
    E4["AES Encryption Key<br/>Integrity Key<br/>IVs<br/>MAC
Keys"]
    E1 --> E2
    E2 --> E3
    E3 --> E4
end

```



Security Access

None

sequenceDiagram

participant Tester

participant ECU

Note over Tester,ECU: UDS Security Access (SID 0x27)

Tester->>ECU: Request Session Control (0x10 xx)

Note right of ECU: Check if requested session is allowed

ECU-->>Tester: Positive Response 0x50 xx P2 P2*

Tester->>ECU: Security Access Request 0x27 01

Note right of ECU: Generate Random Seed

ECU-->>Tester: Seed 0x67 01 + Seed

Note over Tester: Key = f(Seed, Secret)

Note right of ECU: Expected Key = f(Seed, Secret)

Tester->>ECU: Send Key 0x27 02 + Key

Note right of ECU: Compare received key with expected key

ECU-->>Tester: Positive Response 0x67 02

Note right of ECU: Security State Updated Unlocked

